SHR Converter 0.0.2

Signal Hound Spike `.shr` recordings to CSV and native Veusz documents
User and technical manual

William W. Wallace
NRAO/GBO
wwallace@nrao.edu

2026-10-28

Program	Version
`shr_converter.py` (launcher)	0.0.2
`shr_converter_gui.py`	0.0.2
`shr_converter_core.py`	0.0.2
`shr_reader.py` (reader)	0.0.1
`vszh5_writer.py`	0.0.21

Contents

1 Overview **2**
1.1 What is new in 0.0.2 . . . 2

2 Installation **2**

3 The `.shr` format and `shr_reader` **2**
3.1 Byte layout . . . 2
3.2 Enumerations and their confidence . . . 4
3.3 Why detector code 2 is read as Max . . . 4
3.4 Amplitude-grid detection . . . 5
3.5 Validation and salvage . . . 5
3.6 Using the reader from Python . . . 5

4 The window **6**
4.1 Menus and keys . . . 6
4.2 Workflow . . . 6

5 Outputs **8**
5.1 CSV . . . 8
5.2 Veusz document . . . 8

6 Multi-file recordings **8**

7 CSV time decimation **10**

8 Statistics and their derivations **11**
8.1 Frequency of a bin . . . 11
8.2 Power mean, dB mean and the 2.51 dB bias . . . 11
8.3 mV recordings . . . 11
8.4 Standard deviation from running sums . . . 11

8.5 Exact merging of blocks 12
8.6 Spectrogram pixels 12
8.7 Lossless numbers 12

9 Time axes and the even grid **12**

10 Long captures **12**

11 Performance, parallelism, GPU and memory **13**

12 Robustness **15**

13 Command line **15**

14 vszh5_writer 0.0.21 **16**

15 Verification **17**

16 Data gaps **17**

A Options (ConvertOptions) and their defaults **18**

B File-header fields **19**

C Modules and functions used, and why **19**

References **20**

1 Overview

Spike, Signal Hound's analyzer software, records sweeps to `.shr` files: a 472-byte file header, then per sweep a 48-byte sweep header (time stamp, GPS latitude/longitude/altitude, ADC-overflow flag) and the amplitudes as 32-bit floats [1],[2]. The SHR Converter reads these files with its own reader, `shr_reader`, written from the definition in Signal Hound's SDK (`shr_parse.h`) [1] and needing only NumPy. It writes CSV tables and a native Veusz document (`.vszh5`) through the author's `vszh5_writer` [3], optionally rendered by Veusz [14] to a bookmarked PDF and PNGs. Everything is streamed in blocks, so memory is set by a RAM budget and not by the file, and the blocks run in parallel worker processes, with the numerical part on the GPU when CuPy is present.

1.1 What is new in 0.0.2

- **Own reader.** Version 0.0.1 used py-shr-parser 1.0.0 [10], which stops with `ValueError: 2 is not a valid ShrVideoDetector` on SM200C recordings from current Spike releases: their `detector` field holds code 2, which the SDK's enumeration does not list. `shr_reader` keeps every raw code, labels it with a stated confidence and never refuses a file for an unknown value (Section 3).
- **Multi-file recordings** (Spike 3.9.7 [9]) are joined into one recording (Section 6).
- **CSV time decimation**: N sweeps per CSV row, by peak hold, power mean or minimum hold (Section 7).
- **vszh5_writer 0.0.21**: no false blank spectrogram rows from millisecond time-stamp jitter (Section 9).
- GHz axes above 10 GHz, short page titles, wrapped header page; detector shown on the statistics page.

2 Installation

1. `mamba env create -f environment.yml` — Python 3.12; pip installs `requirements.txt` (NumPy, h5py, Qt.py, PySide6, pyqtgraph, psutil, pypdf).
2. `mamba activate shr-converter`
3. Optional: `mamba install -c conda-forge veusz` (or the Veusz installer) to open and render; `mamba install -c conda-forge cupy` for an NVIDIA GPU.
4. `python shr_converter.py --selftest --gui-smoke` — 39 reader checks, 58 engine checks and 12 GUI checks.
5. `python shr_converter.py` starts the GUI. On Windows: `create_environment.bat` once, then `run_shr_converter.bat` (files may be dropped on it) and `run_selftest.bat`.
6. `python shr_converter.py --reader file.shr` prints a file's header, enumerations, notes and amplitude grid without converting anything (`--json`, `--group`, `--grid`).

The launcher imports nothing heavy at its top level: worker processes started with *spawn* re-run the main script's top level, and this keeps Qt (about 50 MB per process) out of them.

3 The `.shr` format and `shr_reader`

3.1 Byte layout

All fields are little-endian and packed (no padding) [1]. A file of S bytes with n bins per sweep holds

$$N_{\text{complete}} = \left\lfloor \frac{S - \texttt{dataOffset}}{48 + 4n} \right\rfloor \tag{1}$$

complete sweep records of $48 + 4n$ bytes each; the SM200C sample ($n = 8384$) has records of 33 584 bytes. The reader parses the header with one `struct` call and maps the records as a NumPy

structured array, so a block of sweeps is one slice of a memory map and no Python loop runs per sweep.

Table 1: File header (472 bytes), generated from shr_reader.HEADER_LAYOUT.

Offset	SDK name	shr_reader name	C type
0	signature	signature	uint16
2	version	version	uint16
4	reserved1	reserved1	uint32
8	dataOffset	data_offset	uint64
16	sweepCount	sweep_count	uint32
20	sweepLength	sweep_length	uint32
24	firstBinFreqHz	first_bin_freq_hz	double
32	binSizeHz	bin_size_hz	double
40	title	title_raw	uint16[128]
296	centerFreqHz	center_freq_hz	double
304	spanHz	span_hz	double
312	rbwHz	rbw_hz	double
320	vbwHz	vbw_hz	double
328	refLevel	ref_level	float
332	refScale	ref_scale	uint32
336	div	div	float
340	window	window	uint32
344	attenuation	attenuation	int32
348	gain	gain	int32
352	detector	detector	int32
356	processingUnits	processing_units	int32
360	windowBandwidth	window_bandwidth	double
368	decimationType	decimation_type	int32
372	decimationDetector	decimation_detector	int32
376	decimationCount	decimation_count	int32
380	decimationTimeMs	decimation_time_ms	int32
384	channelizeEnabled	channelize_enabled	int32
388	channelOutputUnits	channel_output_units	int32
392	channelCenterHz	channel_center_hz	double
400	channelWidthHz	channel_width_hz	double
408	reserved2	reserved2	uint8[64]

Table 2: One sweep record: 48-byte sweep header, then the amplitudes.

Offset	SDK name	shr_reader name	C type
0	timestamp	timestamp_ms	uint64
8	latitude	latitude	double
16	longitude	longitude	double
24	altitude	altitude_m	double
32	adcOverflow	adc_overflow	uint8
33	reserved[15]	reserved	uint8[15]
48	sweep data	amplitude	float32 x sweepLength

3.2 Enumerations and their confidence

Every enumerated field is stored as its raw integer. Labels carry a confidence: *documented* (in `shr_parse.h`), *inferred* (from data, see below) or *unverified* (by analogy, no file seen). Any other code is shown as "unknown (code c)" and becomes a note; it never stops a conversion.

Table 3: Enumerations, generated from shr_reader.ENUM_FIELDS.

Field	Code	Label	Confidence
`ref_scale`	0	dBm	documented
`ref_scale`	1	mV	documented
`window`	0	Nuttall	documented
`window`	1	Flattop	documented
`window`	2	Gaussian	documented
`detector`	0	MinMax	documented
`detector`	1	Average	documented
`detector`	2	Max	inferred
`detector`	3	Min	unverified
`processing_units`	0	Log	documented
`processing_units`	1	Voltage	documented
`processing_units`	2	Power	documented
`processing_units`	3	Sample	documented
`decimation_type`	0	Time	documented
`decimation_type`	1	Count	documented
`decimation_detector`	0	Average	documented
`decimation_detector`	1	Max	documented
`channel_output_units`	0	dBm	documented
`channel_output_units`	1	dBm/Hz	documented

3.3 Why detector code 2 is read as Max

Spike 3.5.7 added max and min detectors [9]; the SDK lists only 0 = MinMax and 1 = Average [1]. Two independent tests on the SM200C sample (460 sweeps × 8384 bins, detector 2) point to Max:

1. **Amplitude grid.** For min/max processing the SM series converts each FFT to 0.01 dB resolution [11]. All 4 999 distinct values of the sample satisfy

$$x = 0.01\,k + 0.00824 \text{ dB}, \qquad k \in \mathbb{Z}, \tag{2}$$

with a largest residual of 7.3×10^{-6} dB (float32 precision). An average detector gives continuous values, so the file came through the min/max path.

2. **Noise shape.** In a noise-only bin the power of one FFT is exponential (unit mean). The maximum M of K FFTs has

$$P(M \le x) = \left(1 - e^{-x}\right)^K, \qquad \mathrm{E}[M] = H_K = \sum_{k=1}^{K} \frac{1}{k} \approx \ln K + \gamma, \tag{3}$$

and $M - \ln K$ tends to the Gumbel distribution (standard deviation $\pi/\sqrt{6} \approx 1.283$, skewness +1.14). With $Y = 10 \log_{10} M$ and $\mathrm{d}Y = \frac{10}{\ln 10}\frac{\mathrm{d}M}{M}$,

$$\sigma_Y \approx \frac{10}{\ln 10}\,\frac{\pi/\sqrt{6}}{\ln K + \gamma} = \frac{4.343 \times 1.283}{6.238 + 0.577} = 0.82 \text{ dB} \quad (K = 512), \tag{4}$$

skewed to the right. The sample's noise bins give $\sigma = 0.745$ dB and skewness +0.51, against 0.77 dB and +0.50 simulated for the max of 512 FFTs. A minimum gives $\sigma_Y = 5.57$ dB for every K, and a power average gives $\sigma_Y \approx 4.343/\sqrt{K}$ with negative skew; neither fits.

The full analysis is in `docs/design/SM200C_SHR_PARSER_BRIEF_0.0.1.md`. Code 3 = Min is assumed by analogy, and what a MinMax (code 0) file stores is not known; both are among the questions to Signal Hound (`docs/SignalHound_support_letter_2026-10-28.md`).

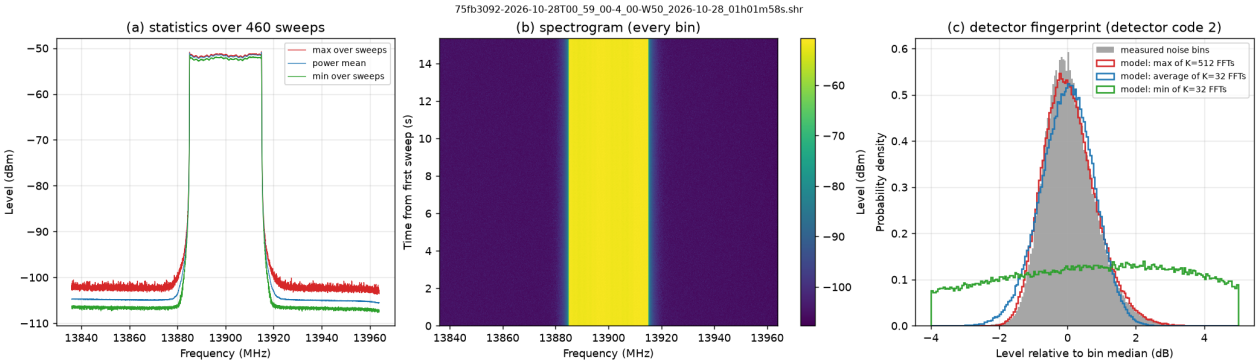


Figure 1: SM200C sample: (a) max, power mean and min over 460 sweeps; (b) spectrogram; (c) noise distribution against detector models (max of K = 512 FFTs fits).

3.4 Amplitude-grid detection

`detect_amplitude_grid` scales the distinct values by the step $q = 0.01$ dB and takes their common fractional part as a circular mean,

$$\phi = \frac{1}{2\pi} \arg\left(\frac{1}{m}\sum_{j=1}^{m} e^{2\pi i\,(x_j/q \bmod 1)}\right) \bmod 1, \qquad r_j = \left|\left(\frac{x_j}{q} - \phi + \tfrac{1}{2}\right) \bmod 1 - \tfrac{1}{2}\right| q, \tag{5}$$

so an offset near 0 (values just below and just above whole steps) is not averaged to $q/2$. The values are on the grid when $\max_j r_j < 5 \times 10^{-4}$ dB and there are at least 16 distinct values. The converter then notes that fixed-decimal CSV output moves values by up to half a step; lossless output is unaffected.

3.5 Validation and salvage

- Signature other than `0xAA10`, impossible sizes or a header shorter than 472 bytes: `ShrFormatError`.
- Version above 2: read with the version-2 layout, with a note. `dataOffset` larger than 472 is honoured.
- `sweepCount` = 0 (recording not closed): the complete sweeps present are read.
- `sweepCount` larger than N_{complete} (cut short): N_{complete} sweeps are read.
- Complete sweeps beyond `sweepCount`, and trailing bytes after the last complete sweep: ignored, with a note.
- Every note reaches the GUI, the log, `_header.csv` and the header page.

3.6 Using the reader from Python

- `open_recording(path or list)` returns a `ShrFile` or, for several files, a `ShrFileSet`; both have `n_sweeps`, `n_bins`, `header`, `frequencies_hz()`, `read(start, stop, step, col0, col1)` (a `SweepBlock` of amplitudes, time stamps, GPS and overflow), `get_sweep(k)`, `iter_blocks()` and `close()`.
- Objects pickle as path and sweep count, so a worker process opens its own memory map; `close()` releases the map, which Windows needs before a file can be moved or deleted.
- `write_shr()` writes a valid file (tests and synthetic recordings).

4 The window



Figure 2: Dark theme, a synthetic 3-hour capture inspected: input list and settings (left), preview (centre), log (bottom).

4.1 Menus and keys

File Add files (Ctrl+O), Add folder (Ctrl+Shift+O), Recent inputs, Remove (Del), Clear, Output folder, Open output folder (Ctrl+Shift+D), Save/Load settings (JSON), Save log, Quit.

Edit Copy file info, copy/clear log, **Free memory** (Ctrl+Shift+M).

View **Theme: System / Light / Dark**, toggle light/dark (Ctrl+T), auto-range (Ctrl+0), panels, toolbar, full screen (F11).

Convert Inspect (Ctrl+I), Convert all (F5), Convert selected (Shift+F5), Stop (Esc), Show plan (Ctrl+P), Self-test.

Veusz Open the `.vszh5` in Veusz (Ctrl+Shift+Z), Render to PDF/PNG (Ctrl+R), Open PDF.

Help Manual (F1), keyboard shortcuts, About (author, licence, every loaded version), About Qt.

4.2 Workflow

1. Add files. Each header is read at once (sweeps, bins, span, length, detector); notes report cut-short recordings, unknown codes and the 0.01 dB grid.
2. Settings: *CSV > Combine N sweeps per row* with its detector (Section 7); *Selection > Join the files of one recording* (Section 6).
3. **Inspect** runs the complete statistics pass without writing any file — the same code as Convert — and shows the preview. The *File info* tab shows the plan (Figure 4).
4. **Convert**: one worker pool serves the whole batch; each file's preview arrives when it is done. **Stop** ends after the current block and deletes that file's partial outputs. Every file of a joined recording shows its status.

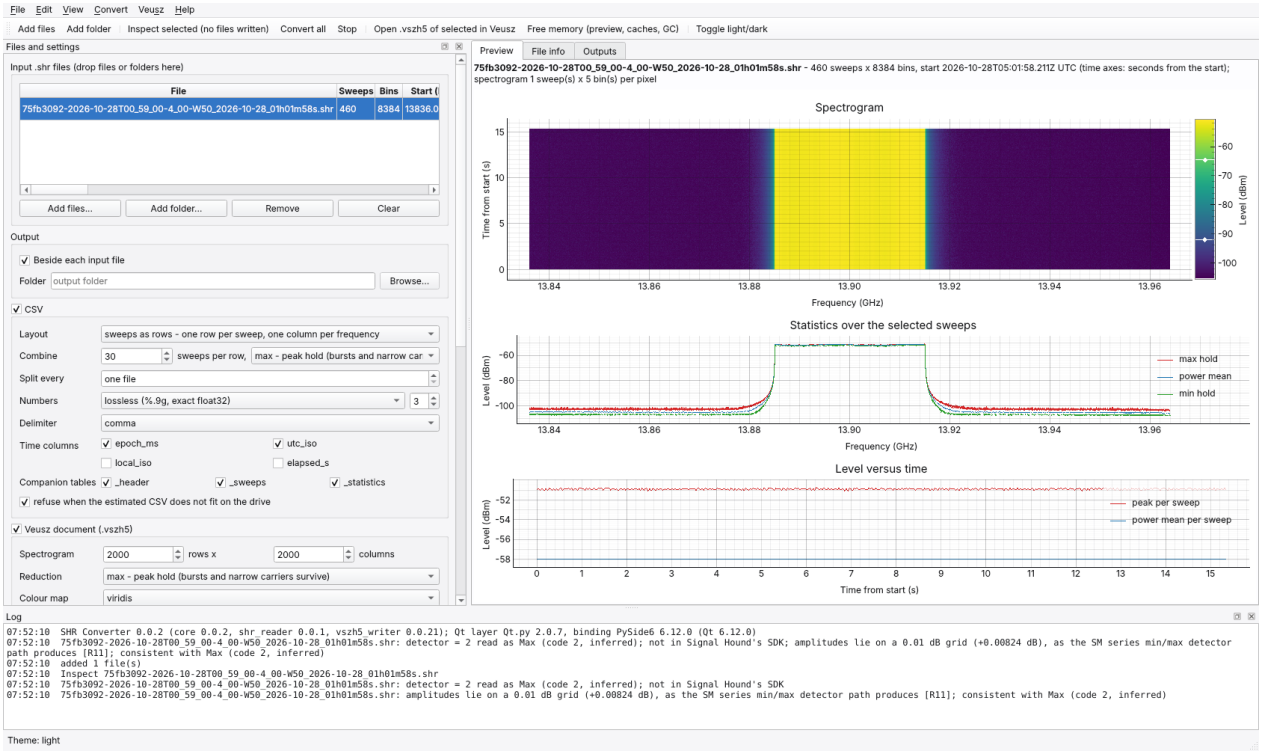


Figure 3: Light theme (View > Theme, or Ctrl+T to toggle).

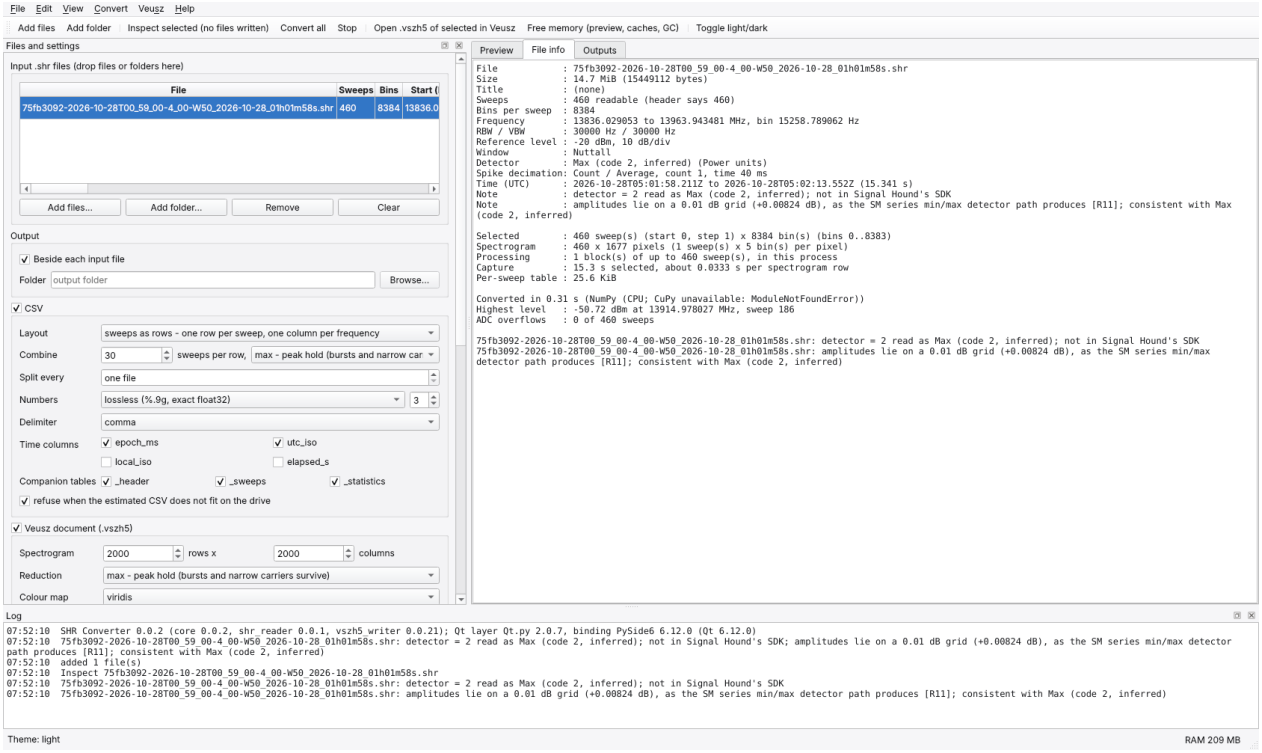


Figure 4: File info: header, plan, CSV estimate, seconds per spectrogram row and free disk space.

5 Outputs

5.1 CSV

`sweeps_as_rows` one row per sweep: index, time columns, GPS, overflow, then one column per frequency (header: the bin frequencies in Hz). The default.

`frequencies_as_rows` one row per bin: frequency, then one column per sweep. Needs every selected sweep of a column block at once, so the file is read once per block (the plan says how often).

`long` one row per sample: index, time columns, frequency, amplitude.

With `csv_decimate` > 1 a `sweeps_combined` column follows the time columns. Companion tables: `_header.csv` (every header field with its unit and enumeration label, derived values, CSV parts with their time spans, the files of a joined recording, versions), `_sweeps.csv` (per sweep: time, GPS, overflow, peak, peak frequency, power mean) and `_statistics.csv` (per bin: max hold, min hold, power mean, dB mean, standard deviation, count). Files are written to private `.part` names and published only when complete; a CSV that another program holds open (Excel on Windows) is kept under a time-stamped name. Excel's limits (16 384 columns, 1 048 576 rows [6]) are reported by the plan.

5.2 Veusz document

Page	Title (PDF bookmark)
`spectrogram`	Spectrogram
`statistics`	Spectrum statistics
`selected_sweeps`	Selected sweeps
`level_vs_time`	Level versus time
`gps_track`	GPS track
`file_header`	File header

6 Multi-file recordings

Since Spike 3.9.7 a long recording continues in a new, sequentially numbered file when the maximum file size is reached [9],[12]. The naming scheme is not documented, so `group_recordings` groups files by content:

1. Files are sorted by their first time stamp.
2. A file joins the previous group when these fields are equal: `sweep_length`, `first_bin_freq_hz`, `bin_size_hz`, `ref_scale`.
3. and it starts no later than

$$t_{\text{first}}^{(j)} - t_{\text{last}}^{(j-1)} \le \max\left(10\ \text{s},\ 50\,\tilde{\Delta}\right), \tag{6}$$

   $\tilde{\Delta}$ being the median sweep interval of the previous file.
4. Differences in `center_freq_hz`, `span_hz`, `rbw_hz`, `vbw_hz`, `detector`, `processing_units`, `window`, `ref_level`, `attenuation`, `gain`, `decimation_type`, `decimation_detector`, `decimation_count`, `decimation_time_ms`, `channelize_enabled`, `channel_output_units` are reported, not refused.

A `ShrFileSet` numbers the sweeps of J files continuously: with N_i sweeps in file i, sweep k_j of file j has the recording-wide index

$$k = O_j + k_j, \qquad O_j = \sum_{i<j} N_i, \tag{7}$$

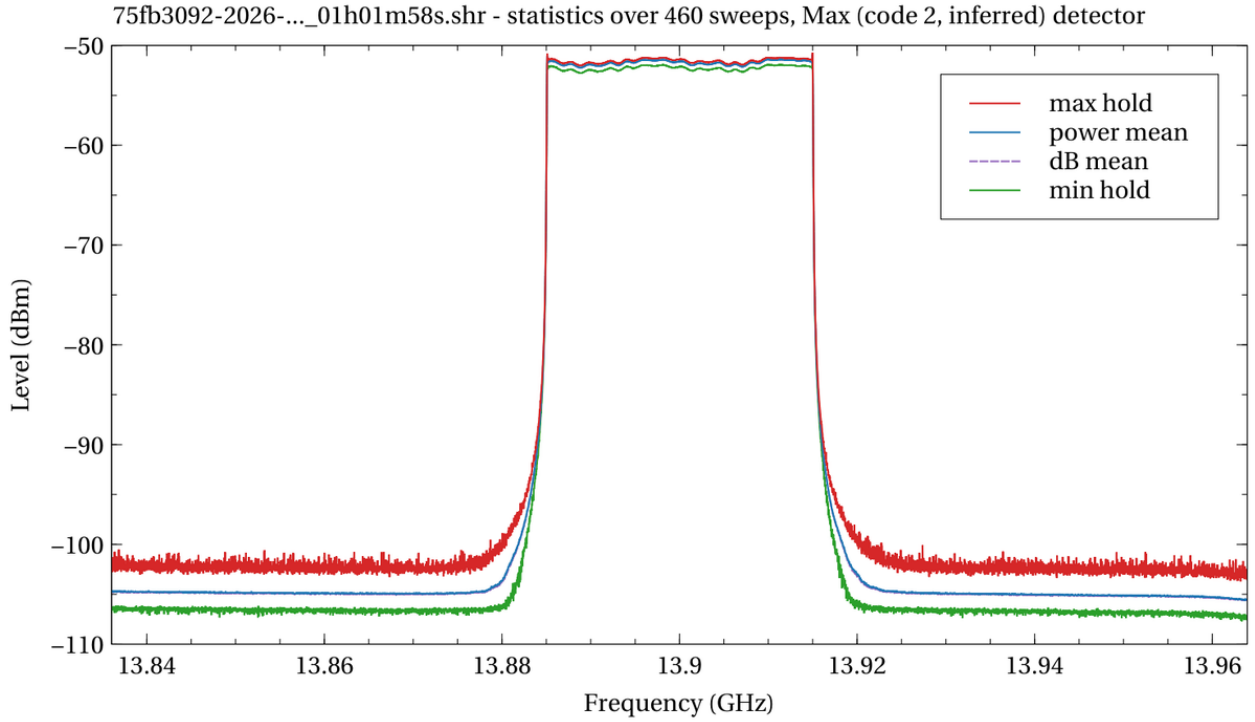


Figure 5: Statistics page of the SM200C sample (460 sweeps, detector code 2 shown as Max, inferred); frequency axis in GHz.

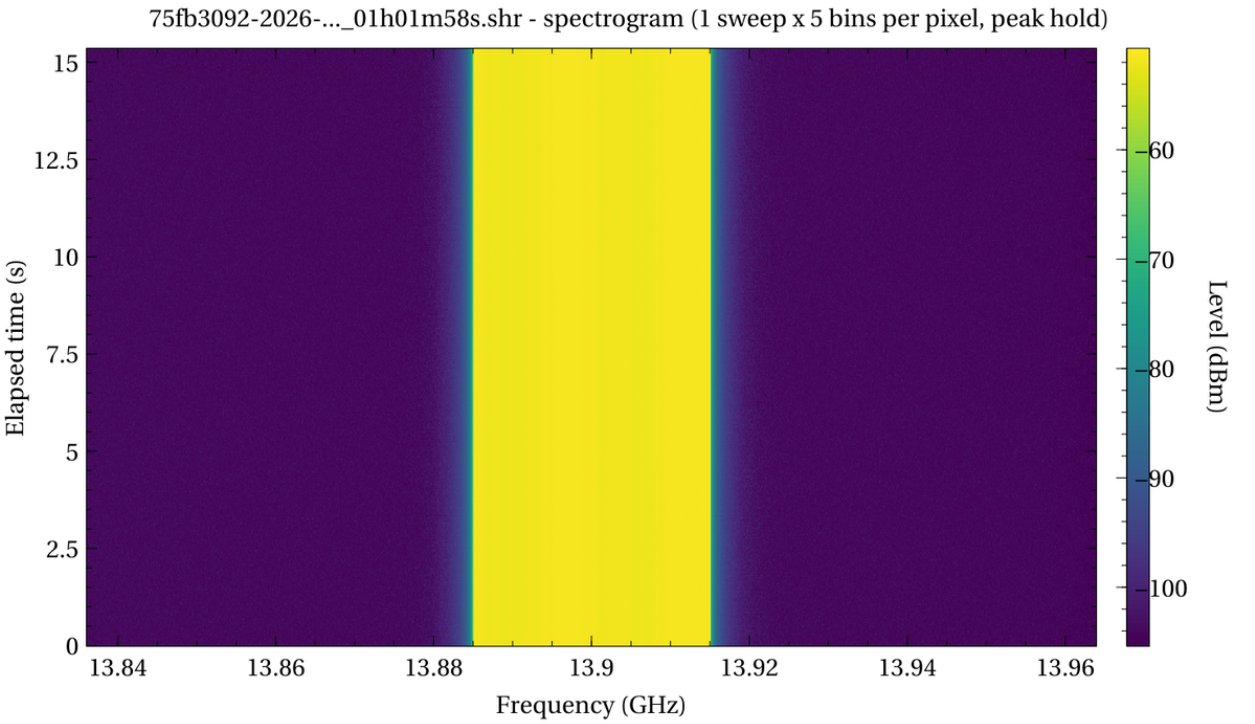


Figure 6: Spectrogram page of the SM200C sample: every row present (no false blank rows with writer 0.0.21).

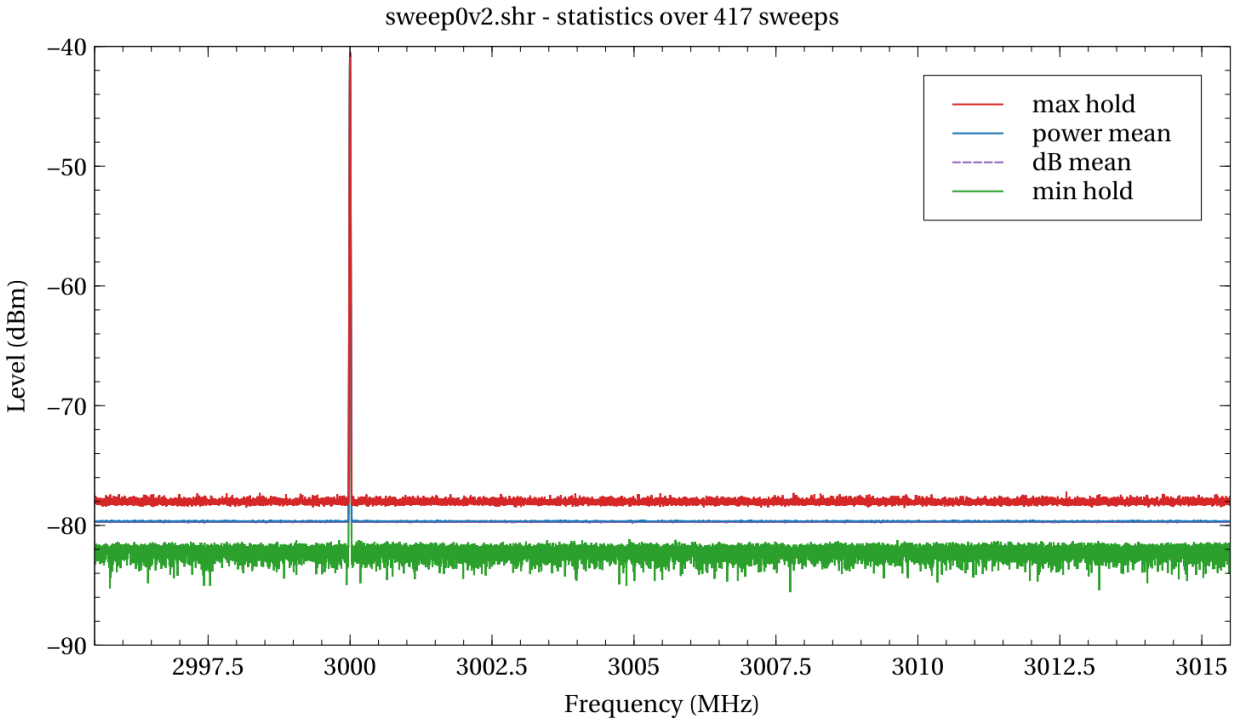


Figure 7: Statistics page of Spike's own test file sweep0v2.shr (417 sweeps): max hold, power mean, dB mean, min hold.

and a read that crosses a file boundary is split and joined transparently. Outputs are named `<first file>_joined.*`; `_header.csv` lists every source file, and gaps between files are noted. Files that do not group are converted on their own (`--join` with `--info` shows the grouping).

7 CSV time decimation

At the SM200C sample's rate a 3-hour recording is about 10.1 GiB of `.shr`; lossless CSV needs about 12 bytes per value, so roughly three times as much. `csv_decimate` $= N$ combines every N selected sweeps into one CSV row (one column in the `frequencies_as_rows` layout). With G_r the sweeps of row r (blocks aligned to multiples of N; the last group may be shorter):

$$y_r^{\max}(f) = \max_{k \in G_r} x_k(f), \qquad y_r^{\min}(f) = \min_{k \in G_r} x_k(f), \tag{8}$$

$$y_r^{P}(f) = 10 \log_{10}\left(\frac{1}{|G_r|} \sum_{k \in G_r} 10^{x_k(f)/10}\right) \ \text{[dBm]}, \qquad y_r^{V}(f) = \sqrt{\frac{1}{|G_r|} \sum_{k \in G_r} x_k(f)^2} \ \text{[mV]}. \tag{9}$$

Detector	Use
`max`	peak hold (bursts and narrow carriers survive)
`power_mean`	power mean (linear power, or RMS for mV)
`min`	minimum hold (noise floor)

Each row carries the index, time stamp and GPS fix of the *first* sweep of its group, ADC overflow if any sweep of the group overflowed, and $|G_r|$ in `sweeps_combined`. Only the main CSV is decimated: the statistics, the spectrogram, `_sweeps.csv` and the `.vszh5` always use every selected sweep.

Table 6: A 3-hour recording at 29.92 sweeps/s x 8384 bins (the SM200C sample's settings), sweeps_as_rows.

N	CSV rows	Time per row	Lossless CSV (estimate)
1	323,134	0.03 s	30.3 GiB
10	32,314	0.33 s	3.0 GiB
30	10,772	1.00 s	1.0 GiB
300	1,078	10.03 s	103.7 MiB

8 Statistics and their derivations

Let $x_k(f)$ be the level of bin f in sweep k, $k = 1 \dots N$.

8.1 Frequency of a bin

$$f_i = f_0 + i\,\Delta f, \qquad f_0 = \texttt{firstBinFreqHz},\ \Delta f = \texttt{binSizeHz}, \tag{10}$$

as the SDK defines the bin axis [1]. py-shr-parser's plotting helpers [10] use linspace$(f_c - S/2, f_c + S/2, n)$ instead; on `sweep0v2.shr` ($\Delta f = 1220.703125$ Hz, $n = 16\,386$, $S = 20$ MHz) the two differ by up to one bin at the top, because $(n-1)\,\Delta f = 20.0012$ MHz $\neq S$. Frequency axes switch to GHz from 10 GHz up, before Veusz would print scientific tick labels.

8.2 Power mean, dB mean and the 2.51 dB bias

$$\bar{P}(f) = 10\log_{10}\Big(\frac{1}{N}\sum_{k=1}^{N} 10^{x_k(f)/10}\Big) \quad \text{[dBm]}, \tag{11}$$

$$\bar{L}(f) = \frac{1}{N}\sum_{k=1}^{N} x_k(f) \quad \text{[dBm]}. \tag{12}$$

For noise the dB mean reads low [4],[13]. With a Rayleigh envelope the power p is exponential with mean μ, $\rho(p) = \mu^{-1}e^{-p/\mu}$, and with $u = p/\mu$

$$\mathrm{E}[\ln p] = \ln\mu + \int_0^\infty \ln u\, e^{-u}\,\mathrm{d}u = \ln\mu + \Gamma'(1) = \ln\mu - \gamma, \tag{13}$$

so

$$\mathrm{E}[10\log_{10} p] - 10\log_{10}\mu = -\frac{10\,\gamma}{\ln 10} = -2.5068\ \text{dB}, \qquad \gamma = 0.5772156649\dots \tag{14}$$

(a 10^7-sample simulation gave -2.5067 dB). The converter therefore reports both: the power mean is the physical mean power, the dB mean matches Spike's log averaging. With a Max detector each x_k is already a maximum over many FFTs (Section 3), so these statistics describe the recorded max-hold traces.

8.3 mV recordings

$$V_{\mathrm{rms}}(f) = \sqrt{\frac{1}{N}\sum_{k} x_k(f)^2} \quad \text{[mV]}. \tag{15}$$

8.4 Standard deviation from running sums

$$s^2 = \frac{S_2 - S_1^2/N}{N-1}, \qquad S_1 = \sum_k x_k,\ S_2 = \sum_k x_k^2. \tag{16}$$

In float64 the cancellation error is about $\varepsilon\,\bar{x}^2/s^2 \approx 10^{-16} \times 80^2/2^2 \approx 2 \times 10^{-13}$ (relative), negligible for dBm levels.

8.5 Exact merging of blocks

Each block returns max, min, S_1, S_2, $\sum 10^{x/10}$ and N per bin; the file's values follow exactly: maxima of maxima, minima of minima, sums of sums, e.g. $\bar{P}_{A\cup B} = 10\log_{10}\big((\Sigma_A+\Sigma_B)/(N_A+N_B)\big)$. Parallel and serial conversions give identical results (self-test).

8.6 Spectrogram pixels

A pixel covers n_r sweeps $\times$ n_c bins. Peak hold is the block maximum; the power mean is

$$\bar{P}_{ij} = 10\log_{10}\Big(\frac{1}{n_r n_c}\sum_{k\in r_i}\sum_{f\in c_j} 10^{x_k(f)/10}\Big) = 10\log_{10}\Big(\frac{1}{n_r}\sum_{k\in r_i}\Big[\frac{1}{n_c}\sum_{f\in c_j} 10^{x_k(f)/10}\Big]\Big), \tag{17}$$

computed as the column mean per sweep, then summed over the sweeps of the row (exact, since every sweep of a row has the same n_c). A row may span two worker blocks; its partial sums are added by the main process.

8.7 Lossless numbers

Binary32 has $p = 24$ significand bits; a decimal string round-trips with [5],[16]

$$d = \lceil p\log_{10} 2\rceil + 1 = \lceil 7.22\rceil + 1 = 9 \tag{18}$$

significant digits, hence `%.9g`. Every value of every layout was compared with the file's float32 values bit for bit (and with py-shr-parser [10] on `sweep0v2.shr`, which it can read).

9 Time axes and the even grid

Time stamps are whole milliseconds since 1970 UTC from the recording PC's clock [2]. The `.vszh5` time axis is UTC or local date/time, elapsed seconds or sweep index (`auto`: date/time for captures of 60 s or more). Veusz draws images with uneven axes through a slow non-linear path (on a 3-hour spectrogram 30.3 s and 3.2 GB, against 2.7 s and 0.31 GB on an even grid), so the rows are put on an even grid by `vszh5_writer.regularize_rows` (0.0.21). With row centres c_k, intervals $d_k = c_{k+1} - c_k$, median $\tilde{d}$ and tolerance $\tau = 0.25$:

$$\text{ordinary}_k : d_k \le (1+2\tau)\,\tilde{d}, \qquad \Delta = \text{mean}\{d_k : \text{ordinary}_k\}, \tag{19}$$

$$s_0 = 0, \qquad s_{k+1} = s_k + \begin{cases} 1 & \text{ordinary}_k \\ \max\big(1, \text{rint}(d_k/\Delta)\big) & \text{otherwise} \end{cases} \tag{20}$$

Row k goes to slot s_k; empty slots are blank (NaN) rows and show a pause. Why the mean: a step Δ' that differs from the true mean interval μ advances each row by μ/Δ' slots, so the rounding skips a slot about every $\Delta'/|\mu - \Delta'|$ rows. Spike's whole-millisecond stamps make the SM200C intervals 32–35 ms (mean 33.35 ms, median 33 ms); version 0.0.20, which took its step from the median, left a false blank row about every 47 sweeps. With the mean of the ordinary intervals no slot is skipped, and a real 2-s pause still gives 60 blank rows. Rows that are not strictly increasing are spread evenly instead (times approximate).

10 Long captures

- **CSV split by time**: `_part001.csv`, …, one per N minutes of recording, each with the header line; slices follow the running maximum of the time stamps, so a clock step back never sends a sweep to an earlier file (and the step is noted).
- **CSV decimation** (Section 7) and **joined files** (Section 6).

- **Disk check**: the CSV size is estimated and the conversion refuses to start when the drive cannot hold it.
- **Per-sweep table**: 57 bytes per sweep, memory-mapped from a private temporary folder when larger than a quarter of the RAM budget.
- **Thinned series**: level versus time and GPS in the `.vszh5` are reduced to `time_series_max_points` (100 000) — peak hold for the peak, power mean for the band level, the frequency of the strongest sweep, any overflow — so a single burst is never averaged away.
- **Time per row**: 2000 rows over 3 h is 5.4 s per row; the plan states it.

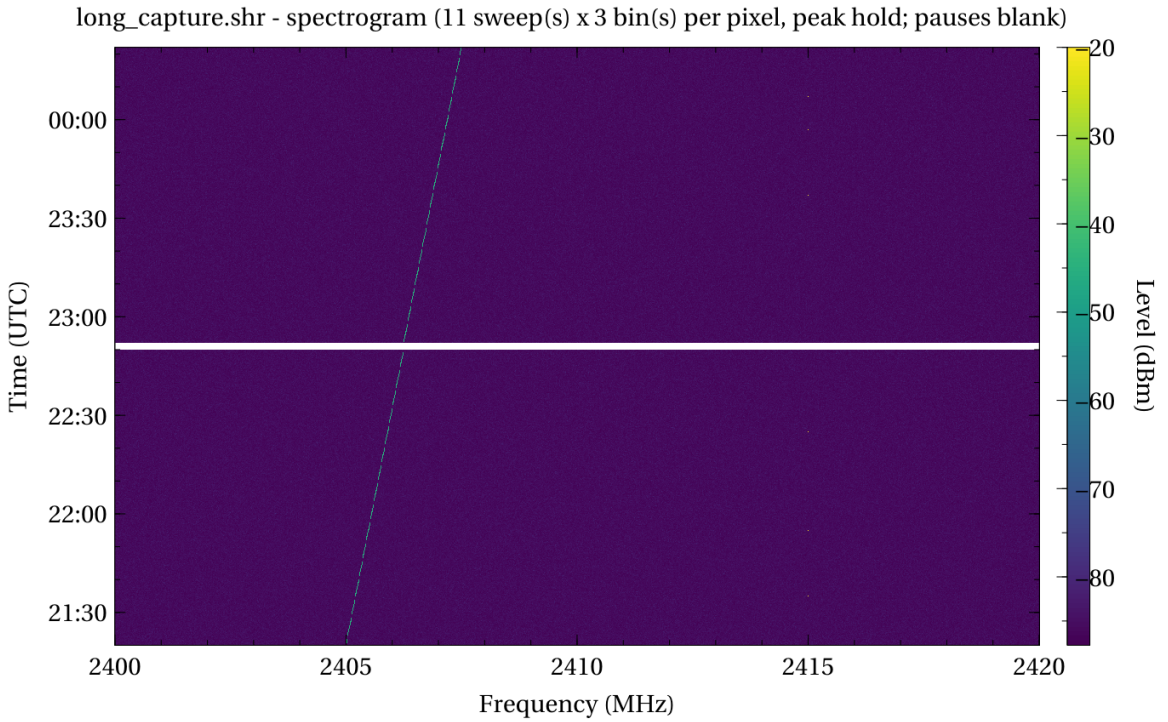


Figure 8: Synthetic 3-hour capture: drifting carrier, a 2 ms burst every 10 min at 2415 MHz, a 2-minute pause (blank band).

11 Performance, parallelism, GPU and memory

- Blocks of sweeps are read (one memory-mapped slice each), formatted and reduced in worker processes (*spawn*, never fork a GUI), at most 2×workers blocks in flight, merged in order [7]. Default workers: CPUs −1 (at most 61 on Windows).
- Block size: $B = \lfloor (M - M_{\text{stats}})/(s\, n_c\, \beta) \rfloor$ with budget M, $s = 2w + 1$ slots, n_c bins and β bytes per sample in flight (30 for `sweeps_as_rows`); the statistics work in sub-blocks of 2 M samples, so their float64 temporaries do not grow with the block. With `csv_decimate` $= N$, B is rounded down to a multiple of N.
- GPU: CuPy [8] runs the statistics and spectrogram reduction per sub-block; any GPU error repeats the block on the CPU (tested with a stand-in). Text formatting stays on the CPU.
- Edit > Free memory: garbage collection, CuPy pools, `malloc_trim` / `EmptyWorkingSet`; previews are pickled to disk, one in memory.

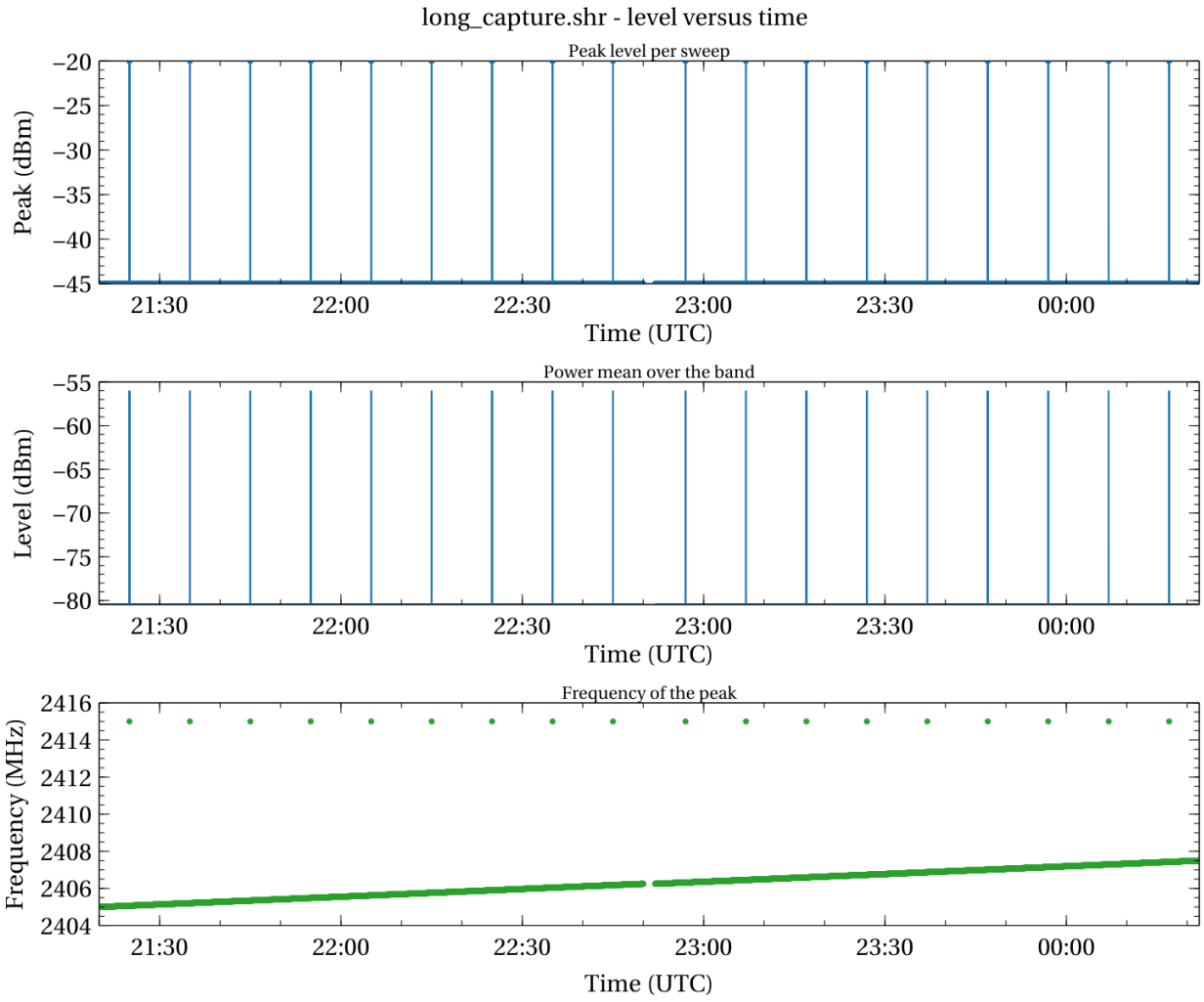


Figure 9: Level versus time of the same capture: every burst present, the pause visible.

Table 7: Measured with tests/long_capture_check.py, tests/memory_check.py and tests/sm200c_rate_check.py (docs/validation).

Case (2 CPU cores, no GPU)	Data	Time	Peak memory
3 h, 2 sweeps/s x 4001 bins, 1.5 ms jitter, 2 workers, with the Veusz PDF/PNG render	331 MiB to 981 MiB CSV, 7 files	27.9 s	865 MiB
3 h, 50 sweeps/s x 512 bins (540 000 sweeps), 2 workers	1.1 GiB to 3.1 GiB CSV	58.9 s	560 MiB
3000 x 65 536 bins, RAM budget 256 MB, 2 workers	375 MiB to 1.1 GiB CSV	21.4 s	417 MiB
SM200C rate: 2 joined files x 10 min (35 902 sweeps x 8384 bins), csv_decimate 30, 2 workers	1.1 GiB to 114 MiB CSV (100.5 MiB/s of SHR)	11.4 s	477 MiB

12 Robustness

- The reader's validation and salvage rules (Section 3) turn cut-short, unclosed or unusual files into notes instead of errors; unknown enumeration codes never stop a conversion.
- Memory maps are closed explicitly after every block and file, so Windows can move or delete the input.
- Every output is written to a private `.part` and published atomically; Stop, an error or a refused disk check leaves no partial file.
- Published files get the usual permissions (writer fix of the 0600 mode, since 0.0.20).

13 Command line

```
python shr_converter.py --cli [options] files-or-folders
```

Option	Meaning
`inputs`	.shr files or folders (folders: every *.shr inside)
`-o, --output-dir`	output folder (default: beside each input)
`--no-csv`	do not write the CSV files
`--layout`	CSV layout: sweeps_as_rows (default), frequencies_as_rows or long
`--fixed`	fixed decimals instead of lossless %.9g
`--delimiter`	CSV delimiter: comma (default), semicolon or tab
`--time-columns`	comma list of epoch_ms, utc_iso, local_iso, elapsed_s
`--no-companions`	skip _header/_sweeps/_statistics.csv
`--split-minutes`	new CSV file every N minutes of recording (0 = one file)
`--csv-decimate`	combine every N selected sweeps into one CSV row (1 = every sweep)
`--csv-decimate-detector`	how –csv-decimate combines sweeps: max (peak hold, default), power_mean or min
`--join`	convert the files of a multi-file Spike recording as one (grouped by content)
`--no-disk-check`	do not refuse when the CSV may not fit
`--no-vszh5`	do not write the Veusz document
`--image-size`	maximum spectrogram ROWSxCOLUMNS

Option	Meaning
`--reduction`	spectrogram pixels: max (peak hold, default), power_mean or min
`--colormap`	Veusz colour map, e.g. viridis, magma, heat
`--z`	data | reference | veusz | MIN,MAX
`--time-axis`	auto (date/time from 60 s on), utc, local, elapsed or index
`--sweeps-to-plot`	selected-sweeps page: first,last,peak and/or sweep indices
`--render`	render PDF (bookmarked) and PNG with Veusz
`--png`	PNG pages when rendering: first (default), all or none
`--veusz`	Veusz program (else VEUSZ_EXE, PATH, usual folders)
`--sweeps`	sweep selection START:STOP:STEP (STOP exclusive)
`--freq`	frequency range FMIN:FMAX in MHz
`-j, --workers`	worker processes (0 = CPUs - 1)
`--ram-mb`	RAM budget for blocks
`--compute`	statistics on auto (CuPy GPU when present), cpu or gpu (falls back to CPU)
`--series-points`	maximum points of the level-vs-time and GPS series in the .vszh5
`--sweep-table`	per-sweep table in RAM or memory-mapped on disk (auto: by size)
`--info`	print header, plan and notes only
`--selftest`	run the self-test and exit
`--real-file`	also check this real .shr in the self-test
`--version`	show program's version number and exit

Examples:

- `python shr_converter.py --cli --join --csv-decimate 30 D:\captures` — every recording in the folder, files of one recording joined, 30 sweeps per CSV row (peak hold).
- `python shr_converter.py --cli --info --join *.shr` — headers, notes and the grouping only.
- `python shr_converter.py --reader --group *.shr` — the reader's own view of the files.

14 vszh5_writer 0.0.21

0.0.21 is 0.0.20 with one fix in `regularize_rows()` (Section 9); no public name, parameter, position or default of 0.0.5, 0.0.10, both 0.0.18 releases, 0.0.19 or 0.0.20 changed. 0.0.20 had joined the two releases both numbered 0.0.18 (R&S FSP logger 0.9.0: render fixes; AB Power Meter Monitor 1.6.0: `VeuszAppender`) with 0.0.19, and added the 0600 file-mode fix, `png_background`, date-axis images, `block_reduce` and `regularize_rows`. No other program calls `regularize_rows()`.

Table 9: Details: docs/WRITER_0.0.21_NOTES.md and docs/validation/writer_compatibility_0.0.21.

Program	Own suite with 0.0.21	Note
R&S FSP logger 0.9.0	44/44 (PySide6; Veusz 4.2.1)	set the About test's expected writer version to 0.0.21
AB Power Meter Monitor 1.7.0	45 PASS, 0 FAIL	none
SA Time Stability Tools 0.0.3	7/7 and 22/22 in Veusz 4.2.1	none

Table 9: Details: docs/WRITER_0.0.21_NOTES.md and docs/validation/writer_compatibility_0.0.21.

Program	Own suite with 0.0.21	Note
QuickSyn package 0.0.10	44 pass, 3 pinned checks fail	same with 0.0.19 and 0.0.20: version literal and pre-0.0.15 temporary name
EMF Envelope FFT 0.0.4, RFI evaluator 0.0.5	API check only	they use the 0.0.13/0.0.14 API, unchanged

15 Verification

`shr_reader.py --selftest`: 39 checks (header size and field offsets, write/read round trip bit-exact, UTF-16 title, detector 2 and unknown codes kept with notes, versions 1 and 3, `dataOffset` 1024, salvage of unclosed and cut-short files, bad signature refused, file sets across boundaries, grouping, pickling, 0.01 dB grid detection, time stamps going backwards). `--selftest`: 58 engine checks with a real file (every CSV layout bit-exact with 1 and 2 workers; statistics, spectrogram and sub-block merging against direct NumPy; joins; decimation by all three detectors; cut-short files; mV; GPU stand-in and fallback; split, disk check, disk-mapped table, thinning; locked file; cancel). `--gui-smoke`: 12 offscreen GUI checks including real worker processes and a joined, decimated conversion. `pytest tests`: 34 tests with 30 sub-tests, real renders in Veusz 4.2.1, Spike's own `sweep0v2.shr` and the SM200C sample (shipped in `tests/data`). Logs: `docs/validation`.

16 Data gaps

Item	Status	Confidence
Real Spike files	sweep0v2.shr (v2, dBm) and the SM200C sample (v2, detector 2) read bit-exactly	High
Detector code 2 = Max	inferred from the 0.01 dB grid and the noise shape; not in the SDK	Medium
Detector code 3 = Min	by analogy; no file seen	Low
MinMax (code 0) contents	which array a MinMax file stores is not known	Unknown
Multi-file recordings	grouped by content; Spike's naming not seen	Medium
Limit-line-triggered recordings	not seen; skipped sweeps would appear as pauses	Unknown
Channelized recordings	read as stored, not interpreted	Unknown
Spike time decimation	which instant a decimated sweep's time stamp marks is not known	Unknown
mV recordings	synthetic only	Medium
Title encoding	UTF-16LE per the SDK; no titled file seen	Medium
GPU	stand-in only; no CUDA device here	Medium
Windows	designed for it (spawn, explicit map close, locked CSV), tested on Linux	Medium

The open format questions are in `docs/SignalHound_support_letter_2026-10-28.md`.

A Options (ConvertOptions) and their defaults

Field	Default
output_dir	""
write_csv	True
csv_layout	"sweeps_as_rows"
csv_number_format	"lossless"
csv_decimals	3
csv_delimiter	"comma"
csv_time_columns	('epoch_ms', 'utc_iso')
csv_header_table	True
csv_sweeps_table	True
csv_statistics_table	True
csv_split_minutes	0.0
csv_decimate	1
csv_decimate_detector	"max"
check_disk_space	True
write_vszh5	True
vszh5_compress	True
image_max_rows	2000
image_max_columns	2000
image_reduction	"max"
colormap	"viridis"
z_mode	"data"
z_min	-120.0
z_max	-20.0
time_axis	"auto"
selected_sweeps	"first,last,peak"
stats_max_points	200000
time_series_max_points	100000
page_spectrogram	True
page_statistics	True
page_selected_sweeps	True
page_level_vs_time	True
page_gps	True
page_header	True
render	False
render_png	"first"
render_dpi	150
veusz_exe	""
sweep_start	0
sweep_stop	0
sweep_step	1
freq_min_hz	0.0
freq_max_hz	0.0
join_files	False
workers	0
ram_budget_mb	1024
compute	"auto"
sweep_table_storage	"auto"
flush_after_file	True

B File-header fields

Field	Unit	Meaning
signature		file signature (0xAA10)
version		SHR format version
data_offset	byte	offset of the first sweep
sweep_count		sweeps according to the header
sweep_length	bin	bins per sweep
first_bin_freq_hz	Hz	frequency of bin 0
bin_size_hz	Hz	bin spacing
center_freq_hz	Hz	centre frequency
span_hz	Hz	span
rbw_hz	Hz	resolution bandwidth
vbw_hz	Hz	video bandwidth
ref_level	dBm or mV	reference level
ref_scale		reference scale (dBm or mV)
div	dB	display division
window		RBW window
attenuation	dB	attenuation setting (as stored by Spike)
gain		gain setting (as stored by Spike)
detector		video detector (Max/Min codes are not in the SDK)
processing_units		video processing units
window_bandwidth	bin	window bandwidth in bins (RBW / bin spacing)
decimation_type		time averaging type
decimation_detector		time averaging detector
decimation_count		sweeps per output trace (count decimation)
decimation_time_ms	ms	time averaging interval (time decimation)
channelize_enabled		channelizer enabled (1) or not (0)
channel_output_units		channelizer units
channel_center_hz	Hz	channelizer centre
channel_width_hz	Hz	channelizer width

C Modules and functions used, and why

Module	Used	Why
shr_reader (0.0.1)	ShrHeader, ShrFile, ShrFileSet, group_recordings, detect_amplitude_grid	reads every header and sweep (memory-mapped, one vectorised read per block); picklable, so each worker opens its own map [1]
struct	Struct('<HHIQIIdd256s...64s')	the 472-byte file header in one call [1]
numpy	memmap with a structured dtype, float32/float64 arrays, ufunc.reduceat, add.reduceat, percentile	sweep records, statistics, block and CSV decimation, per-sweep table on disk
cupy (optional)	same array calls on the GPU	statistics and spectrogram reduction per sub-block [8]
concurrent.futures	ProcessPoolExecutor (spawn, max_tasks_per_child)	parallel blocks, bounded and in order [7]

Module	Used	Why
`h5py via vszh5_writer`	VeuszWriter, image_page, xy_page, grid_page, text_page	native .vszh5 without Veusz
`vszh5_writer (0.0.21)`	block_reduce, regularize_rows, render_pages	thinning, even spectrogram grid, PDF/PNG with bookmarks [3]
`Qt.py + PySide6`	QtCore, QtGui, QtWidgets	the GUI through the abstraction layer [15]
`pyqtgraph`	ImageItem, ColorBarItem, PlotDataItem	preview plots
`psutil`	Process().memory_info()	memory read-out and tests
`pypdf (via the writer)`	PdfWriter.add_outline_item	PDF bookmark tree
`ctypes`	malloc_trim (Linux), EmptyWorkingSet (Windows)	return freed memory to the OS

References

References

[1] Signal Hound, "SHR parsing example" (`shr_parse.h`, `shr_parse.cpp`), in *Signal Hound Software Development Kit*, release of 2026-10-05. https://signalhound.com/software/signal-hound-software-development-kit-sdk/

[2] Signal Hound, ".shr file format specification," BB Series Discussions forum, May–Aug. 2018. https://signalhound.com/?p=158701

[3] W. W. Wallace, "vszh5_writer.py — Veusz HDF5 document writer," v0.0.21, NRAO/GBO, 2026 (shipped with this program).

[4] Hewlett-Packard, "Measuring Noise and Noise-like Digital Communications Signals with a Spectrum Analyzer," Application Note 1303, lit. no. 5966-4008E. https://www.hpmemoryproject.org/an/pdf/an_1303.pdf

[5] IEEE, "IEEE Standard for Floating-Point Arithmetic," IEEE Std 754-2019, pp. 1–84, Jul. 2019, doi: 10.1109/IEEESTD.2019.8766229. https://doi.org/10.1109/IEEESTD.2019.8766229

[6] Microsoft, "Excel specifications and limits." https://support.microsoft.com/en-us/office/1672b34d-7043-467e-8e27-269d656771c3

[7] Python Software Foundation, "concurrent.futures — Launching parallel tasks," Python 3.12. https://docs.python.org/3.12/library/concurrent.futures.html

[8] Preferred Networks, "CuPy documentation." https://docs.cupy.dev/

[9] Signal Hound, "Technical Software and Hardware Change Log" (Spike), v4.0.17, Sep. 2026 (max/min detectors 3.5.7; multi-file sweep recording 3.9.7; limit-line triggered recording 4.0.17). https://signalhound.com/sigdownloads/Spike/change_log.txt

[10] T. Schmitz, "py-shr-parser: Python library for parsing Signal Hound SHR files," v1.0.0, WiSE Lab, Carnegie Mellon Univ., 2025. https://github.com/WiseLabCMU/py-shr-parser

[11] Signal Hound, "SM200A/B/C – SM435B/C Product Manual," Jan. 2025 (min/max processing at 0.01 dB resolution). https://signalhound.com/sigdownloads/SM200A/SM-User-Manual.pdf

[12] Signal Hound, “Spike Software Documentation” (Sweep Record and Playback). https://signalhound.com/sigdownloads/Spike/docs/html/index.html

[13] Keysight Technologies, “Spectrum Analysis Basics,” Application Note 150, 5952-0292. https://www.keysight.com/us/en/assets/7018-06714/application-notes/5952-0292.pdf

[14] J. Sanders, “Veusz — a scientific plotting package.” https://veusz.github.io/

[15] M. Ottosson *et al.*, “Qt.py,” v2.0.7. https://github.com/mottosso/Qt.py

[16] R. Regan, “Number of digits required for round-trip conversions,” Exploring Binary. https://www.exploringbinary.com/number-of-digits-required-for-round-trip-conversions/